Agent Postmortem Generator

Part 6

Graph Generation Engine

Purpose: Transform deterministic analysis results into canonical execution graph artifacts for report rendering.

Design Document

This module is part of the Agent Postmortem Generator architecture and must remain framework agnostic, deterministic, and local-first.

2. Executive Overview

The Graph Generation Engine turns deterministic analysis output and canonical trace references into execution graph artifacts that can be embedded in the postmortem report. It does not read raw traces, parse framework-specific event objects, or decide what the failure means. It receives already-normalized data and renders a stable graph representation of how the execution unfolded.

The module exists because incident reports are easier to trust when the reader can see the execution shape, not just read prose. Engineers want to know what happened first, where branching occurred, which steps retried, where the terminal failure emerged, and how the failure propagated. The graph module makes that structure visible without introducing probabilistic layout logic or hidden analysis.

In the system pipeline, this module sits after the Analysis Engine and before the Postmortem Generation / Output layer. The Analysis Engine identifies the important events and relationships. The Graph Generation Engine converts those facts into graph-ready artifacts, usually Mermaid plus a canonical JSON representation for downstream rendering and testing.

3. Problem Statement

Raw execution traces are noisy, large, and structurally inconsistent across agent frameworks. Even after normalization, the report still needs a compact way to explain execution flow without collapsing the trace into prose. The exact problem here is to produce a graph artifact that is stable, deterministic, and faithful to the normalized trace and analysis results.

This problem is hard because agent runs are not simple linear scripts. They branch, retry, invoke tools, resume after failures, and sometimes stop mid-stream. Traces may contain repeated labels, missing timestamps, partial failures, or parallel child activity. If the graph engine invents structure to make the diagram look nicer, the report becomes untrustworthy. If it is too literal, the graph becomes unreadable. The module has to walk that line without pretending uncertainty is certainty.

The MVP needs this module because a text-only postmortem is not enough for most debugging sessions. The graph is the fastest way to confirm the failure chain, the first meaningful fault, and the propagation path. It is a trust-building artifact, not decoration.

4. Goals

- Generate a graph artifact from canonical trace and analysis data without consulting raw framework objects.
- Preserve the execution order, parent-child structure, retry structure, and analysis-identified fault path.
- Emit deterministic node and edge ordering for identical inputs.
- Expose evidence references on graph elements so downstream report sections can cite them.

- Produce a graph format that the output layer can embed directly in markdown and later export to richer formats.
- Keep the implementation small enough for a 2 to 3 engineer team to maintain without specialist tooling.

5. Non-Goals

- Parsing raw agent traces or validating the input JSON schema.
- Normalizing framework-specific events into the canonical trace model.
- Inferring root cause, first fault, or remediation steps.
- Generating report prose, summaries, or recommendations.
- Rendering image-based graphs such as PNG, SVG, or interactive web canvases in the MVP.
- Cross-run comparison, drift detection, or visual diffing between executions.
- Layout optimization, rank tuning, or probabilistic graph beautification.
- Framework-specific graph semantics leaking into the canonical graph vocabulary.

Future versions can add interactive expand/collapse behavior, visual layout engines, image exports, and graph diff views. Those are natural follow-ons, but they are not required for the first release.

6. Responsibilities

Owns	Must Never Own
Graph vocabulary: node types, edge types, groups, annotations, and diagnostic metadata.	Raw trace parsing, schema repair, or framework adapter logic.
Construction of a deterministic execution graph from canonical trace and analysis data.	Failure analysis, root cause selection, or blame attribution.
Stable serialization to graph-friendly output such as Mermaid and structured JSON.	Report authoring, markdown sectioning, or PDF export.
Validation of graph-specific invariants such as node uniqueness, reference integrity, and ordering stability.	Heuristic guessing about missing events or unsupported structure.
Highlighting the analysis-identified fault path and propagation path.	Deciding which path is important without explicit analysis input.

The boundary is deliberately narrow. If something changes the meaning of the failure, it belongs in analysis. If something changes the shape of the report, it belongs in postmortem generation. If something changes how framework events are understood, it belongs in adapters or normalization. This module only turns stable facts into stable graph artifacts.

7. Inputs

Input	Required	Notes
Canonical Trace Model	Yes	Normalized, validated, framework-agnostic event set with stable event identifiers and ordering metadata.
Analysis Result Model	Yes	Contains the analysis-identified first meaningful fault, failure chain, propagation path, and any explicit causal links.
Graph Options	Yes	Small, versioned options only: output format, rank direction, and optional display flags.
Report Context	No	Optional document metadata such as run name or report title for labeling only.
Evidence References	Yes	Per-node or per-edge references that point back to canonical trace events or analysis facts.

Validation rules:

- Every referenced event id in the analysis result must exist in the canonical trace model.
- Node identifiers must be stable and unique within the graph document.
- Edge endpoints must resolve to known nodes.
- If timestamps are present, they must be comparable, but the engine must still work when some events have no timestamp.
- Unsupported graph options must fail fast rather than silently falling back.

Assumptions:

- The canonical trace model already removed framework-specific semantics that this module should not need.
- The analysis result has already selected the important fault path or provided a deterministic ordered candidate list.
- The caller wants a reproducible graph artifact, not an animated or interactive layout.

Malformed input handling:

- Hard failures: missing required event references, broken node/edge integrity, unsupported output type, or self-contradictory analysis references.

- Soft degradation: missing optional labels, absent timestamps, truncated tail markers, or incomplete display metadata. These should result in warnings, not guessed facts.
- The module must never invent a node or edge simply to make the graph look complete.

8. Outputs

Output Artifact	Type	Purpose
GraphDocument	Structured model	Internal canonical graph object consumed by serializers and downstream report assembly.
Mermaid Graph Text	Human-readable text	Primary MVP output for embedding in markdown reports and documentation.
Graph Diagnostics	Structured warnings/errors	Explains missing optional context, unsupported shapes, or partially degraded output.
Graph Metadata	Structured metadata	Graph version, source trace ids, counts, and determinism hash for snapshot testing.
Optional JSON Export	Structured text	Machine-readable form for future integrations and regression fixtures.

Human-readable artifacts are intentionally limited to text-based graph output. The module does not own final report rendering, typography, or page layout. Those are handled by the output layer and whatever PDF pipeline sits on top of it.

The most important output contract is stability. Given the same trace and analysis inputs, the module must produce the same graph document and the same Mermaid text byte-for-byte, subject only to explicitly versioned configuration changes.

9. Internal Design

The module should be organized into a small set of subcomponents with clear boundaries. Do not over-abstract this. The graph engine is not a general graph library; it is a narrow execution-graph generator for one product domain.

Subcomponent	Responsibility
Input Resolver	Loads the canonical trace references and analysis facts needed for graph generation.
Topology Builder	Creates the graph nodes and edges from ordered events, execution scopes, and analysis-

	highlighted paths.
Annotation Layer	Attaches fault markers, propagation markers, retry markers, and evidence references.
Deterministic Validator	Checks node uniqueness, edge integrity, and ordering stability before serialization.
Serializer	Converts the graph document into Mermaid text and the optional structured export.

Workflow:

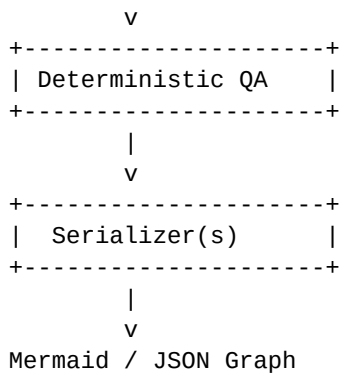
```
1. Receive canonical trace and analysis result.
2. Resolve the graph scope and extract the relevant event subset.
3. Build a stable node set from the execution order in the canonical trace.
4. Add edges for parent-child structure, temporal progression, and analysis-provided
causal links.
5. Apply annotations for first fault, terminal failure, retries, and propagation.
6. Validate graph invariants and produce diagnostics.
7. Serialize to Mermaid and/or JSON using deterministic ordering.
```

Important internal decisions:

- The module should build one canonical graph document and derive all serializer outputs from that single structure.
- The graph is not a full semantic model of the agent run. It is a view over the execution facts that matter for debugging.
- The module must use stable identifiers derived from canonical event ids and graph scope, not from randomized UUIDs.
- The module should prefer explicit repeated nodes or retry-group metadata over literal cycles if the underlying execution repeats work.
- No layout algorithm should be allowed to alter semantic meaning. Layout is a serializer concern, not an analysis concern.

```
High-level flow:

Canonical Trace + Analysis Result
        |
        v
+---------------------+
|   Input Resolver    |
+---------------------+
        |
        v
+---------------------+
|  Topology Builder   |
+---------------------+
        |
        v
+---------------------+
| Annotation Layer    |
+---------------------+
        |
```



10. Data Model

Entity	Key Fields	Why It Exists
GraphDocument	graph_id, source_trace_id, analysis_id, version, nodes, edges, groups, diagnostics	Single container for the entire graph artifact and its metadata.
GraphNode	node_id, event_id, kind, label, status, order_index, scope_id, evidence_refs	Represents one stable execution event or annotated execution concept.
GraphEdge	edge_id, from_node_id, to_node_id, kind, label, evidence_refs, style_hint	Represents a relationship between two graph nodes.
GraphGroup	group_id, kind, label, member_node_ids, order_index	Keeps related nodes together in a readable, deterministic way.
GraphAnnotation	target_id, annotation_type, label, severity, evidence_refs	Attaches semantic meaning to nodes or edges without changing the underlying topology.
GraphDiagnostics	code, severity, message, related_ids, recoverable	Explains warnings and recoverable output degradation.

Core relationships:

- A GraphDocument contains many GraphNodes and GraphEdges.
- A GraphNode may belong to one GraphGroup for display purposes, such as a retry group or a tool chain scope.
- GraphAnnotations reference exactly one node or edge target and never replace the base topology.
- GraphDiagnostics are attached to the GraphDocument, not to arbitrary output strings.

Why this model exists:

- It creates a stable boundary between analysis facts and graph serialization.
- It makes snapshot testing and regression review straightforward.
- It keeps graph generation testable without requiring a renderer or browser runtime.
- It allows future renderers to reuse the same semantic graph document.

11. Algorithms / Processing Logic

Processing should be deterministic and should never depend on probabilistic ranking or live external services.

1. Resolve the relevant trace scope from the analysis result. The graph engine should only include events that belong to the execution path or context selected by the analysis layer.
2. Create graph nodes in canonical event order. If timestamps are missing or equal, fall back to the canonical trace ordering and then stable event ids.
3. Create the primary execution spine from the root event through the terminal failure event. This spine must reflect the actual execution order in the normalized trace.
4. Add child and sibling structure where the trace indicates nested tool calls, model calls, agent steps, or retry attempts. These relationships must come from canonical trace structure, not label similarity.
5. Overlay analysis-provided relationships such as first meaningful fault and propagation chain. Do not infer new causal links beyond the analysis output and direct execution structure.
6. Mark retries and repeated attempts as distinct nodes with retry-group metadata. Do not compress them into a single abstract node if that would hide the actual execution history.
7. Validate the resulting graph for missing references, duplicate identifiers, and unresolved endpoints.
8. Serialize nodes and edges in a stable order, using a versioned serializer contract so future changes can be snapshot-tested.

Edge cases:

- Missing timestamps: use canonical event order and stable ids.
- Parallel branches: preserve sibling order by canonical index, not by label or label length.
- Repeated labels: qualify labels with event type and stable short id if needed for readability.
- Truncated traces: render the last visible node and mark the graph as truncated rather than guessing the missing tail.
- Ambiguous analysis output: if the analysis layer provides an ordered candidate list instead of a single fault, show the first candidate and attach an ambiguity diagnostic.
- Cycles in source data: do not create literal cycles in the graph model. Represent repetition through repeated nodes and retry grouping.

Deterministic behavior requirements:

- Equal inputs must produce equal node ids, edge ids, ordering, and serialization text.

- No randomization, hash iteration dependence, or unordered collection traversal may affect output.
- All serializer-specific formatting decisions must be versioned and explicitly documented.

12. Tradeoffs

Tradeoff	Chosen Approach	Reason
Simplicity vs flexibility	One canonical graph model with a small vocabulary.	This keeps the module testable and reduces the chance of meaning drift.
Generality vs specificity	Model the graph for this product's execution semantics rather than all possible graphs.	The product needs trustworthy debugging artifacts, not a general graph framework.
Determinism vs heuristic inference	No heuristic completion or guessed causal edges.	Trust matters more than visual completeness.
Readability vs semantic fidelity	Prefer faithful structure, even when the graph is less visually compact.	If the graph lies less, engineers trust it more.

The preferred design is deliberately conservative. It gives up some visual polish in exchange for stable output, small code size, and fewer failure modes. That is the right trade for an MVP that is supposed to be trusted by experienced engineers.

13. Risks and Failure Modes

Hidden complexity tends to appear in the places humans assume are trivial: retries, partial failures, parallel tool calls, and inconsistent timestamps. This module must expose those cases instead of smoothing them over.

- Ambiguous execution shapes can lead to unreadable graphs if too much context is included.
- Missing or stale analysis references can break graph construction even when the canonical trace is valid.
- Large traces can create wide graphs that are hard to read in markdown even if the data model is correct.
- Graph semantics may drift if later modules start relying on graph node kinds as a proxy for analysis meaning.
- If the serializer becomes too smart, it can introduce non-deterministic ordering or hidden formatting changes.

Unsupported inputs:

- Raw framework-specific traces without normalization.

- Analysis outputs that reference unknown event ids or claim relationships unsupported by the canonical trace.
- Requests for image rendering, interactivity, or advanced layout control in the MVP.

Maintenance risks:

- Adding too many node or edge kinds will make the graph vocabulary harder to reason about.
- Special-casing every framework's quirks will break the framework-agnostic boundary.
- Mixing graph generation with report prose or markdown formatting will create cyclical dependencies and reduce testability.

14. Testing Strategy

This module should be tested through snapshot-friendly fixtures and exact structural assertions. The output is deterministic, so the tests should be too.

- Unit tests: node id stability, edge creation rules, annotation mapping, serializer ordering, diagnostics generation.
- Integration tests: canonical trace plus analysis result to full graph artifact, including Mermaid output.
- Fixture-driven tests: linear trace, branching trace, tool retry trace, truncated trace, missing timestamp trace, ambiguous analysis trace, parallel branch trace.
- Regression protection: exact string snapshots for Mermaid, plus JSON snapshot or structural compare for the graph document.
- Negative tests: unknown event references, duplicate node ids, invalid edge endpoints, unsupported output format, and non-deterministic ordering traps.

A good fixture set should include at least one example where the graph is intentionally ugly but correct. That keeps the implementation honest and prevents layout polish from mutating the semantics.

15. Implementation Checklist

- Define the graph document schema and version it explicitly.
- Define the node, edge, group, annotation, and diagnostics types with stable identifiers.
- Implement the input resolver against the canonical trace model and analysis result model.
- Implement the topology builder for sequence, nesting, retry, and analysis-highlight relationships.
- Implement deterministic ordering rules for nodes, edges, and groups.
- Implement graph validation and diagnostics reporting.
- Implement Mermaid serialization first, then optional JSON export.
- Add fixture-based tests before broadening the graph vocabulary.
- Document the serializer contract and the stability guarantees.

- Wire the module into postmortem generation only after snapshot tests pass.

16. Future Expansion

- SVG or PNG export for richer reports and external sharing.
- Interactive graph views with collapse and expand controls.
- Cross-run graph comparison and drift highlighting.
- Graphviz or ELK-backed layout engines for more advanced visual formatting.
- Graph filters for depth, scope, retry groups, and failure-only views.
- Hover metadata and clickable evidence references in a UI layer.

These expansions are natural because they build directly on the graph document and serializer contract. They should still keep the canonical graph model as the source of truth.

17. Final Recommendation

This design is MVP-ready if the implementation stays limited to one canonical execution graph model, deterministic serialization, and a small number of well-defined node and edge kinds. That is enough to support credible postmortems without dragging the project into graph-tooling theater.

The only simplifications still recommended are to defer image export, interactive layout, and advanced visual styling until the core graph artifact is proven useful in real debugging sessions. The MVP should optimize for trust and maintainability, not visual spectacle.